Università degli studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Analisi di sicurezza e integrità crittografica di un sistema di versionamento distribuito

Tesi di laurea triennale

Relatore

Prof. Vardanega

Laureando

Michele Stevanin

Matricola 2101741

ANNO ACCADEMICO 2025-2026

© Michele Stevanin, Luglio 2026. Tutti i diritti riservati. Tesi di laurea triennale: *Analisi di sicurezza e integrità crittografica di un sistema di versionamento distribuito.*

«Chi non conosce la storia è condannato a ripeterla,
ma chi la conosce è condannato a guardare gli altri che la ripetono.»

— Primo Levi

Sommario

Questa relazione descrive il lavoro svolto durante lo stage curricolare presso Zucchetti S.p.A., della durata di trecentoquattro ore, sotto la supervisione del tutor aziendale Gregorio Piccoli e del tutor accademico Prof. Vardanega.

Lo stage ha riguardato l'analisi della sicurezza di *RVC*$_G$, un sistema di versionamento distribuito sviluppato internamente da Zucchetti S.p.A. come alternativa a *Git*$_G$. A differenza dei sistemi tradizionali, *RVC*$_G$ non richiede un server centrale per la verifica di autenticità dei *commit*$_G$: i *commit*$_G$ vengono distribuiti come archivi firmati, navigabili direttamente tramite filesystem. L'integrità dei contenuti è garantita attraverso la verifica crittografica degli *hash*$_G$ di ogni *commit*$_G$, con l'obiettivo di permettere a qualsiasi utente di accertare autonomamente l'autenticità della *repository*$_G$ ricevuta, indipendentemente dalla fonte.

Il lavoro ha previsto lo studio delle tecnologie crittografiche alla base del sistema — chiavi *SSH*$_G$, firma-digitale e strumenti di cifratura — seguito dalla simulazione di scenari di attacco con diversi livelli di accesso. Per ciascuno scenario sono state individuate le vulnerabilità presenti, analizzate le possibili conseguenze e proposte contromisure concrete.

Organizzazione del testo

Il primo capitolo presenta l'azienda ospitante, introduce il progetto *RVC*$_G$ e illustra le motivazioni che mi hanno portato a scegliere questo stage, con gli obiettivi definiti, la pianificazione e l'analisi dei rischi;

Il secondo capitolo descrive l'organizzazione del lavoro durante il tirocinio, l'ambiente di sviluppo, gli strumenti utilizzati e l'approccio metodologico seguito;

Il terzo capitolo illustra le tecnologie e i concetti teorici alla base del progetto, dalla crittografia-asimmetrica e *SSH*$_G$ fino all'architettura di *RVC*$_G$ e al confronto con *Git*$_G$

Il quarto capitolo definisce il modello di sicurezza che un sistema di versionamento distribuito moderno dovrebbe soddisfare, analizzando le proprietà fondamentali richieste, la gerarchia di fiducia, i livelli di sicurezza configurabili e la gestione del

ciclo di vita delle identità, concludendo con un'analisi del divario rispetto allo stato attuale di *RVC*$_G$

Il quinto capitolo presenta gli scenari di attacco simulati in ambiente controllato, con diversi livelli di accesso — da attaccante esterno fino a compromissione della chiave del capo progetto — e analizza la propagazione degli errori nella catena degli *hash*$_G$

Il sesto capitolo descrive i miglioramenti implementati nel codice sorgente di *RVC*$_G$, tra cui il sistema di configurazione dinamico, la firma *SSH*$_G$ integrata, la verifica dell'integrità della catena e la gestione degli *allowed signers*;

Il settimo capitolo riassume i risultati raggiunti, valuta il grado di soddisfacimento degli obiettivi prefissati e propone una riflessione personale sull'esperienza e sugli sviluppi futuri.

Convenzioni tipografiche

Durante la stesura del testo sono state adottate le seguenti convenzioni tipografiche:

- Gli acronimi, le abbreviazioni e i termini di uso non comune vengono definiti nel glossario, situato alla fine del documento (p. 45);
- I termini presenti nel glossario sono indicati con la notazione: *RVC*$_G$;
- I termini in lingua straniera non di uso comune o appartenenti al gergo tecnico sono evidenziati in *corsivo*;
- I nomi di funzioni, variabili o comandi sono scritti con carattere `monospaziato`;
- I riferimenti bibliografici sono indicati con il numero identificativo della fonte, es. [1];
- I blocchi di codice sorgente sono rappresentati nel seguente modo:

```cpl
proc ReportInfo(FileManifestPersistent fm, bool files:=false)
  if fm.tag <> nil
    ? '    tag:', fm.tag
  end
  if fm.prev <> nil
    ? '   prev:', fm.prev
  end
  if fm.merge <> nil
    ? '  merge:', fm.merge
  end
end
```

Esempio di funzione CPL estratta dal sorgente di *RVC*$_G$.

Indice

1 Introduzione ..... **1.**

1.1 L'azienda ..... 1.
1.2 Il progetto ..... 2.
1.3 Scelta del progetto ..... 2.
1.4 Obiettivi dello stage ..... 3.
1.5 Pianificazione ..... 4.
1.6 Analisi dei rischi ..... 5.

2 Descrizione dello stage ..... **7.**

2.1 Organizzazione del lavoro ..... 7.
2.1.1 Rapporto con i tutor ..... 7.
2.2 Ambiente di sviluppo ..... 8.
2.2.1 Strumenti utilizzati ..... 8.
2.3 Approccio metodologico ..... 9.
2.3.1 Ambiente di simulazione ..... 9.

3 Tecnologie e fondamenti teorici ..... **11.**

3.1 Crittografia asimmetrica e firma digitale ..... 11.
3.1.1 La firma digitale in dettaglio ..... 12.
3.1.2 Curve ellittiche: Ed25519 ..... 13.
3.2 SSH e ssh-keygen ..... 13.
3.2.1 Autenticazione con chiavi ..... 13.
3.2.2 ssh-keygen per la firma di file ..... 14.
3.3 AGE ..... 14.
3.4 Sistemi di controllo versione ..... 15.
3.4.1 Storia e evoluzione ..... 15.
3.4.1.1 Sistemi centralizzati ..... 15.
3.4.1.2 Sistemi distribuiti ..... 16.

3.4.2 Git in dettaglio ... 17.
3.4.3 RVC a confronto con Git ... 17.
3.5 RVC: architettura e funzionamento ... 18.
3.5.1 Struttura della repository ... 18.
3.5.2 Il file .sig e la blockchain degli hash ... 18.
3.5.3 Il linguaggio CPL ... 19.
3.5.4 Flusso di un commit ... 20.

4 Requisiti e modello di sicurezza ... 21.

4.1 Sicurezza strutturale nei sistemi di versionamento distribuito ... 21.
4.1.1 Confronto con i sistemi esistenti ... 22.
4.1.2 Applicazione in RVC ... 23.
4.2 Requisiti di sicurezza formali ... 23.
4.2.1 Integrità e ordine verificabile ... 23.
4.2.2 Autenticità e non ripudio ... 24.
4.2.3 Gestione delle identità ... 24.
4.2.4 Sicurezza configurabile ... 25.
4.2.5 Gestione dei branch ... 26.
4.3 Gerarchia di fiducia ... 27.
4.3.1 Amministratore (CapoProgetto) ... 27.
4.3.2 Responsabile di progetto ... 27.
4.3.3 Dipendente ... 28.
4.3.4 Cliente ... 28.
4.3.5 Inizializzazione della repository ... 28.
4.3.6 Inizializzazione di un progetto ... 29.
4.3.7 File amministrativi della repository ... 30.
4.3.8 Catena di fiducia tra progetti ... 30.
4.3.9 Implicazioni di sicurezza della radice pubblica ... 31.
4.4 Livelli di sicurezza configurabili ... 31.
4.4.1 Livello 0 — Aperto ... 32.
4.4.2 Livello 1 — Autenticato ... 32.
4.4.3 Livello 2 — Autorizzato ... 32.
4.4.4 Livello 3 — Verificato ... 32.
4.4.5 Livello 4 — Riservato ... 33.
4.4.6 Gestione dei destinatari nel livello 4 ... 33.
4.5 Gestione delle identità e ciclo di vita delle chiavi ... 34.

4.5.1 Cambio chiave ordinario ..... 34.
4.5.2 Revoca per compromissione ..... 35.
4.5.3 Revoca offline ..... 35.
4.5.4 Successione del responsabile ..... 36.
4.5.5 Compromissione della chiave dell'amministratore ..... 36.
4.6 Gestione dei branch ..... 37.
4.6.1 Archiviazione di branch inutili ..... 37.
4.6.2 Chiusura di branch compromessi ..... 37.
4.6.3 Branch e permessi ..... 38.
4.7 Gap analysis ..... 38.

5 Simulazione scenari di attacco ..... 39.

6 Miglioramenti implementati ..... 41.

7 Conclusioni ..... 43.

7.1 Consuntivo finale ..... 43.
7.2 Raggiungimento degli obiettivi ..... 43.
7.3 Requisiti soddisfatti ..... 43.
7.4 Rischi occorsi e mitigati ..... 44.
7.5 Valutazione personale ..... 44.

Glossario ..... 45.

Elenco delle Figure

1.1 Logo di Zucchetti S.p.A. ..... 1.
3.1 Flusso trasferimento messaggio con uso di chiave pubblica e privata ..... 12.
3.2 Flusso firma di un documento con chiave pubblica e privata ..... 12.
3.3 Comparazione dimensione tra tipologie di chiavi ..... 13.

3.4 Esempio di un file cifrato con AGE . . . . . 15.
3.5 Controllo versioni centralizzato vs distribuito . . . . . 16.
3.6 Struttura del file .sig e la catena degli hash cumulativi . . . . . 19.
3.7 Struttura dei moduli principali di RVC . . . . . 20.

Elenco delle Tabelle

1.1 Obiettivi dello stage. . . . . 3.
1.2 Pianificazione del periodo di stage. . . . . 4.
1.3 Analisi preventiva dei rischi. . . . . 5.
3.1 Confronto tra Git e RVC. . . . . 17.
4.1 Requisiti di integrità e ordine verificabile. . . . . 24.
4.2 Requisiti di autenticità e non ripudio. . . . . 24.
4.3 Requisiti di gestione delle identità. . . . . 25.
4.4 Requisiti di sicurezza configurabile. . . . . 26.
4.5 Requisiti di gestione dei branch. . . . . 26.
4.6 Confronto tra i livelli di sicurezza configurabili. . . . . 34.
7.1 Consuntivo orario finale. . . . . 43.
7.2 Riepilogo dei requisiti soddisfatti. . . . . 44.
7.3 Rischi occorsi con la loro mitigazione. . . . . 44.

Elenco dei Codici Sorgente

0.1 Esempio di funzione CPL estratta dal sorgente di RVC$_G$. vii

Capitolo 1

Introduzione

In questo capitolo viene descritta l'azienda ospitante, introdotto il progetto e illustrate le motivazioni che hanno portato alla scelta di questo stage.

1.1 L'azienda

Zucchetti S.p.A. è un'azienda italiana con sede principale a Lodi che produce soluzioni software, hardware e servizi per aziende, professionisti e associazioni di categoria. Le origini risalgono al 1977, quando lo studio del commercialista Domenico «Mino» Zucchetti realizzò il primo software in Italia per l'elaborazione automatica delle dichiarazioni dei redditi. L'anno successivo, nel 1978, fu fondata formalmente l'azienda.

Oggi il gruppo conta più di 8.000 dipendenti, di cui 2.000 dedicati alla Ricerca e Sviluppo, e serve oltre 700.000 clienti tramite sedi distribuite in tutta Italia e più di 1.650 partner. A livello internazionale è presente con proprie società in diversi paesi europei e oltreoceano, con una rete di oltre 350 partner in 50 paesi. Zucchetti si posiziona oggi come la prima *software house* in Italia per fatturato.

Lo stage è stato svolto presso la sede di Padova e di Noventa Padovana (PD), sotto la supervisione del tutor aziendale Gregorio Piccoli.

Logo di Zucchetti S.p.A.

1.2 Il progetto

Il progetto ha riguardato l'analisi della sicurezza di *RVC*$_G$, un sistema di versionamento distribuito sviluppato internamente da Zucchetti S.p.A. come alternativa a *Git*$_G$. A differenza dei sistemi tradizionali, *RVC*$_G$ non richiede un server centrale: i *commit* vengono distribuiti come archivi firmati, navigabili direttamente tramite filesystem. L'integrità dei contenuti è garantita attraverso la verifica crittografica degli *hash*$_G$ di ogni *commit*, con l'obiettivo di permettere a qualsiasi utente di accertare autonomamente l'autenticità della *repository* ricevuta, indipendentemente dalla fonte di distribuzione.

L'autenticazione e la firma dei *commit* avvengono tramite chiavi *SSH*$_G$, rendendo ogni modifica crittograficamente attribuibile al suo autore. Questo approccio distingue *RVC*$_G$ non solo per l'architettura distribuita, ma anche per le garanzie di autenticità che offre rispetto ai sistemi di versionamento convenzionali.

La versione di *RVC*$_G$ fornita per lo stage è una versione di sviluppo deliberatamente priva di alcuni meccanismi di sicurezza, con l'obiettivo di permettere uno studio autonomo delle vulnerabilità e la progettazione di soluzioni originali. Questo approccio ha consentito di affrontare il problema della sicurezza senza vincoli architetturali predefiniti, producendo analisi e implementazioni indipendenti.

1.3 Scelta del progetto

Ho scelto questo progetto per l'interesse verso la sicurezza informatica e la crittografia applicata, temi che avevo già incontrato durante il percorso universitario ma che non avevo mai avuto l'occasione di approfondire in un contesto reale. L'analisi di un sistema in uso aziendale, con l'obiettivo di individuare vulnerabilità concrete e proporre miglioramenti, rappresenta un'opportunità difficilmente replicabile in ambito accademico.

La natura del lavoro — che combina studio teorico delle tecnologie crittografiche e simulazione pratica di scenari di attacco — mi ha convinto che fosse il progetto più adatto per concludere il percorso triennale con un contributo tangibile.

1.4 Obiettivi dello stage

Gli obiettivi dello stage sono stati definiti in accordo con il tutor aziendale e classificati secondo tre livelli di priorità:

- **Obbligatori** (O): obiettivi il cui soddisfacimento è vincolante per la riuscita del progetto;
- **Desiderabili** (D): obiettivi non strettamente necessari ma dal riconoscibile valore aggiunto;
- **Facoltativi** (F): obiettivi che rappresentano un ulteriore contributo non competitivo.

Codice	Descrizione
O01	Studio delle tecnologie *SSH*$_G$, *AGE*$_G$ e *RVC*$_G$
O02	Analisi del sistema *RVC*$_G$ e individuazione delle vulnerabilità
O03	Simulazione di attacchi senza possesso di credenziali
O04	Simulazione di attacchi con chiave-privata compromessa
O05	Produzione della documentazione tecnica e della relazione finale
D01	Simulazione di attacchi con chiave del capo progetto compromessa
D02	Gestione dei *signers* per *branch* diversi all'interno della *repository*
D03	Studio e implementazione delle tecniche di recovery delle credenziali
F01	Analisi delle possibilità di adozione di una struttura monorepo o polirepo in *RVC*$_G$
F02	Progettazione e sviluppo di un sistema di *repository* cifrato con distribuzione dei permessi

Obiettivi dello stage.

1.5 Pianificazione

Il lavoro è stato organizzato su otto settimane per un totale di 304 ore, suddivise tra studio delle tecnologie, analisi della sicurezza, sviluppo e documentazione.

Settimana	Ore	Attività
1	32	Studio di *SSH*$_G$, crittografia-asimmetrica, firma-digitale e *AGE*$_G$
2	40	Studio di *RVC*$_G$: architettura, formato dei *commit*$_G$
3	40	Analisi del codice sorgente *CPL*$_G$ e individuazione delle vulnerabilità
4	40	Simulazione scenari di attacco senza credenziali e con chiave compromessa
5	40	Implementazione miglioramenti: configurazione, firma *SSH*$_G$, verifica integrità
6	32	Gestione signers, allowed_signers, verifica catena completa
7	40	Simulazione attacchi avanzati e sviluppi futuri
8	40	Completamento e revisione della relazione finale

Pianificazione del periodo di stage.

1.6 Analisi dei rischi

Prima dell'avvio del progetto è stata condotta un'analisi preventiva dei rischi, al fine di individuare le possibili criticità e predisporre le opportune contromisure.

Descrizione	**Contromisura**	**Probabilità** **Impatto**
Codice sorgente di *RVC*$_G$ non disponibile nella fase iniziale, con impossibilità di analisi *white-box*$_G$	Analisi *black-box*$_G$ tramite osservazione del comportamento esterno e dei file prodotti	Alta Alto
Linguaggio *CPL*$_G$ proprietario senza documentazione pubblica, con curva di apprendimento elevata	Studio della documentazione interna fornita dall'azienda e confronto diretto col tutor	Alta Medio
Comportamenti silenziosi del sistema in caso di errore, che rendono difficile il debug	Aggiunta sistematica di istruzioni di debug nel codice sorgente durante l'analisi	Media Alto
Vulnerabilità individuate già risolte nella versione interna, rendendo il lavoro ridondante	Verifica periodica col tutor aziendale sull'allineamento tra la versione di test e quella interna	Media Medio

Analisi preventiva dei rischi.

Capitolo 2

Descrizione dello stage

In questo capitolo viene descritta l'organizzazione del lavoro durante il tirocinio, l'ambiente di sviluppo utilizzato, gli strumenti adottati e l'approccio metodologico seguito.

2.1 Organizzazione del lavoro

Lo stage si è svolto in presenza presso le sedi di Zucchetti S.p.A., con rare eccezioni in modalità *smart working* nei periodi in cui l'azienda era impegnata in un trasferimento di sede. La prima settimana si è tenuta presso gli uffici di via Giovanni Cittadella a Padova; le settimane successive presso la sede principale di Noventa Padovana (PD), dove ho trascorso la maggior parte del tirocinio.

L'orario di lavoro era strutturato su due fasce giornaliere: dalle 9:00 alle 13:00 e dalle 14:00 alle 18:00, per un totale di otto ore al giorno. Questa organizzazione ha permesso di alternare sessioni di studio teorico al mattino con attività pratiche nel pomeriggio, sfruttando la pausa di mezzogiorno per consolidare i concetti appresi.

2.1.1 Rapporto con i tutor

Il confronto con i tutor ha avuto modalità diverse a seconda del ruolo.

Il **tutor aziendale**, Gregorio Piccoli, era disponibile quasi quotidianamente per chiarimenti, revisioni del lavoro svolto e indicazioni sui passi successivi. Le riunioni non seguivano una cadenza fissa ma avvenivano su richiesta, ogni volta che si presentava un problema tecnico rilevante o si raggiungeva un risultato degno di discussione. Questo approccio ha favorito un dialogo continuo e ha permesso di adattare il percorso in modo flessibile all'avanzamento del lavoro.

Il **tutor accademico**, il Professor Prof. Vardanega, è stato invece coinvolto con cadenza settimanale tramite scambio di email. Le comunicazioni riguardavano principalmente l'avanzamento dello stage rispetto alle attese previste e la verifica dell'allineamento con gli obiettivi didattici dello stage.

2.2 Ambiente di sviluppo

Per tutta la durata dello stage ho utilizzato un portatile fornito dall'azienda con sistema operativo Windows. La scelta degli strumenti da installare è stata lasciata alla mia discrezione, senza vincoli imposti dall'azienda, il che ha permesso di configurare un ambiente di lavoro ottimale per le esigenze del progetto.

L'accesso ai sistemi aziendali includeva anche i modelli linguistici (*LLM*) disponibili localmente nell'infrastruttura di Zucchetti, utilizzati come supporto durante le fasi di studio e sviluppo.

2.2.1 Strumenti utilizzati

Gli strumenti principali adottati durante lo stage sono stati:

Visual Studio Code come editor di testo principale, utilizzato sia per la scrittura e modifica del codice sorgente *CPL*$_G$ di *RVC*$_G$, sia per la stesura della documentazione in Typst. L'editor è stato configurato con estensioni per la sintassi *CPL*$_G$ e per la compilazione live dei documenti Typst.

GitHub per il versionamento della tesi e dei file di lavoro personali. Il *repository*$_G$ della relazione finale è ospitato su GitHub con pubblicazione automatica del PDF tramite *GitHub Actions* e *GitHub Pages*.

Typst come sistema di composizione tipografica per la stesura della relazione finale. Typst è un'alternativa moderna a LaTeX che permette di produrre documenti PDF di qualità professionale con una sintassi più accessibile.

Terminale Windows (cmd e PowerShell) per l'esecuzione dei comandi *RVC*$_G$, la gestione dei file e le operazioni di debug. La maggior parte delle operazioni con *RVC*$_G$ avviene tramite interfaccia a riga di comando.

PuTTY come client *SSH*$_G$ per Windows, utilizzato per la gestione delle chiavi *SSH*$_G$ tramite il componente `puttygen`.

Ssh-keygen (OpenSSH per Windows) per la generazione delle chiavi *Ed25519*$_G$, la firma crittografica dei file e la verifica delle firme *SSH*$_G$. Questo strumento è centrale nel meccanismo di sicurezza implementato in *RVC*$_G$.

AGE$_G$ come strumento di cifratura moderno, studiato durante lo stage in preparazione alla fase di progettazione delle *repository* cifrate prevista negli obiettivi desiderabili.

2.3 Approccio metodologico

Il percorso di lavoro ha seguito un andamento alternato tra fasi teoriche e fasi pratiche, con transizioni determinate dall'avanzamento della comprensione del sistema e dalla disponibilità del materiale.

Nella **fase iniziale** l'attenzione era rivolta allo studio delle tecnologie crittografiche — chiavi *SSH*$_G$, firma-digitale, *AGE*$_G$ — attraverso documentazione ufficiale, esempi pratici al terminale e confronto con il tutor aziendale. Parallelamente ho iniziato a esplorare *RVC*$_G$ dall'esterno, analizzandone il comportamento tramite i comandi disponibili e studiando il formato dei file prodotti.

Una volta ottenuto accesso ai **sorgenti *CPL*$_G$**, l'approccio è cambiato: dall'analisi esterna (*black-box*) si è passati all'analisi interna (*white-box*), con lettura sistematica del codice, identificazione delle vulnerabilità e progettazione degli interventi migliorativi. Questa fase ha richiesto anche lo studio del linguaggio *CPL*$_G$, per il quale non esiste documentazione pubblica — la comprensione è avvenuta tramite lettura del codice esistente e consultazione della documentazione interna fornita dall'azienda.

La **fase di implementazione** ha proceduto in parallelo con l'analisi: man mano che venivano individuate vulnerabilità o carenze, venivano progettate e implementate le relative soluzioni, testandole su una *repository* di simulazione appositamente creata.

2.3.1 Ambiente di simulazione

Per riprodurre scenari di attacco in modo controllato e reversibile, ho configurato un ambiente di test dedicato composto da:

- Una **directory di lavoro** (`simulazione/`) contenente i file di un progetto fittizio su cui eseguire le operazioni *RVC*$_G$.
- Una ***repository*$_G$ locale** (`repo/`) dove vengono archiviati i *commit*, separata dalla directory di lavoro.
- Un file `.git/repository.info` che collega la directory di lavoro alla *repository*$_G$, seguendo la convenzione di *RVC*$_G$.
- Un file `allowed_signers` contenente le chiavi pubbliche degli autori autorizzati a firmare i *commit*, utilizzato per i test di verifica dell'integrità.

Questo ambiente ha permesso di simulare scenari realistici — inclusi la manomissione dei file di *commit*, la compromissione delle chiavi *SSH*$_G$ e la verifica della propagazione degli errori nella catena degli *hash*$_G$ — senza rischiare di danneggiare dati reali.

Capitolo 3

Tecnologie e fondamenti teorici

In questo capitolo sono illustrate le tecnologie e i concetti teorici alla base del progetto. Partendo dalla crittografia-asimmetrica, vengono descritti SSH$_G$, AGE$_G$ e i sistemi di controllo versione, fino ad arrivare all'architettura di RVC$_G$.

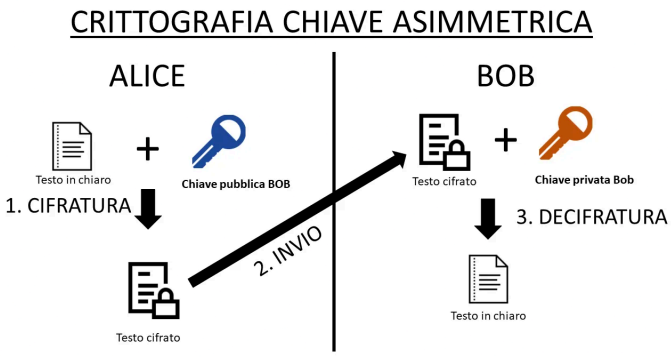
3.1 Crittografia asimmetrica e firma digitale

La crittografia è la disciplina che studia le tecniche per proteggere le informazioni rendendole illeggibili a chi non è autorizzato. Nella sua forma classica, detta *crittografia simmetrica*, mittente e destinatario condividono una stessa chiave segreta per cifrare e decifrare i messaggi. Questo approccio presenta un problema fondamentale: come si trasmette la chiave in modo sicuro prima ancora di poter comunicare in modo sicuro?

La risposta a questo problema arrivò negli anni "70 con la *crittografia asimmetrica*, detta anche *crittografia a chiave pubblica*. L"idea è elegante: ogni utente genera una coppia di chiavi matematicamente collegate tra loro. La **chiave-pubblica** può essere distribuita liberamente a chiunque. La **chiave-privata** deve rimanere segreta e non lasciare mai il dispositivo del proprietario. Le due chiavi sono collegate da una relazione matematica tale per cui ciò che viene cifrato con una può essere decifrato solo con l'altra, ma è computazionalmente impossibile ricavare la chiave-privata a partire da quella pubblica.

Questo meccanismo permette due operazioni fondamentali:

- **Cifratura**: chiunque può cifrare un messaggio usando la chiave-pubblica del destinatario. Solo il destinatario, possedendo la chiave-privata corrispondente, potrà decifrarlo.
- **Firma-digitale**: il mittente cifra un messaggio con la propria chiave-privata. Chiunque, usando la chiave-pubblica del mittente, può verificare che il messaggio provenga effettivamente da lui e non sia stato alterato.



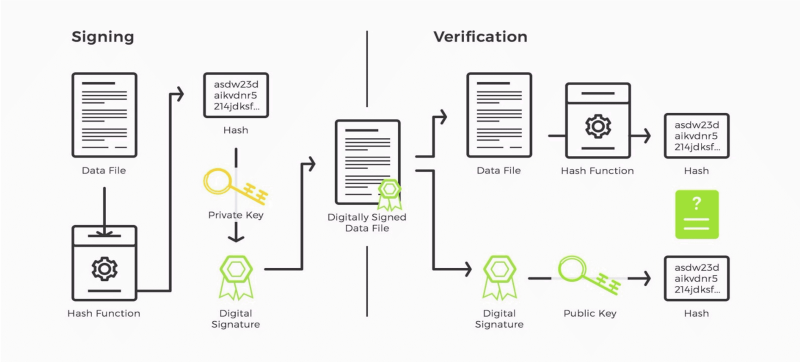
Flusso trasferimento messaggio con uso di chiave pubblica e privata

3.1.1 La firma digitale in dettaglio

La firma-digitale non cifra il messaggio intero — sarebbe inefficiente. Funziona invece in questo modo:

1. Il mittente calcola l'*hash* del messaggio, ovvero una sua impronta digitale di dimensione fissa prodotta da una funzione matematica non invertibile.
2. Il mittente cifra quell'hash con la propria chiave-privata. Il risultato è la firma-digitale.
3. Il destinatario riceve messaggio e firma. Decifra la firma usando la chiave-pubblica del mittente, ottenendo l'hash originale.
4. Il destinatario calcola autonomamente l'hash del messaggio ricevuto e confronta i due valori. Se coincidono, la firma è valida: il messaggio proviene dal mittente dichiarato e non è stato modificato.

La sicurezza di questo meccanismo si basa su due proprietà: la resistenza alle collisioni delle funzioni di $hash_G$ (è praticamente impossibile trovare due messaggi diversi con lo stesso $hash_G$) e l'impossibilità computazionale di produrre una firma valida senza possedere la chiave-privata.



Flusso firma di un documento con chiave pubblica e privata

3.1.2 Curve ellittiche: Ed25519

Gli algoritmi crittografici moderni si basano su problemi matematici computazionalmente difficili. RSA, il primo algoritmo a chiave-pubblica diffuso, si basa sulla difficoltà di fattorizzare numeri molto grandi. Gli algoritmi moderni basati su *curve ellittiche* offrono lo stesso livello di sicurezza con chiavi molto più corte, risultando più veloci ed efficienti.

Ed25519$_G$ è un algoritmo di firma-digitale basato sulla curva ellittica Curve25519. Produce chiavi di 256 bit e firme di 512 bit, con un livello di sicurezza equivalente a RSA con chiavi da 3000 bit. È l'algoritmo raccomandato oggi per la firma *SSH*$_G$ ed è quello utilizzato in questo progetto.

Symmetric Key Size (bits)	RSA and Diffie-Hellman Key Size (bits)	Elliptic Curve Key Size (bits)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	521

Comparazione dimensione tra tipologie di chiavi

3.2 SSH e ssh-keygen

SSH$_G$ è un protocollo di rete che permette di stabilire connessioni sicure tra due macchine attraverso una rete non sicura. Nato come sostituto sicuro di Telnet e rsh, *SSH*$_G$ cifra tutto il traffico e autentica sia il server che il client, impedendo attacchi di tipo *man-in-the-middle*.

3.2.1 Autenticazione con chiavi

SSH$_G$ supporta due modalità principali di autenticazione: tramite password e tramite coppia di chiavi. L'autenticazione con chiavi è considerata più sicura e viene raccomandata in tutti i contesti professionali. Il funzionamento è il seguente:

1. L'utente genera una coppia di chiavi con `ssh-keygen`.
2. La chiave-pubblica viene copiata sul server remoto, nella cartella `~/.ssh/authorized_keys`.
3. Al momento della connessione, il server invia una sfida cifrata con la chiave-pubblica dell'utente.
4. Il client dimostra di possedere la chiave-privata corrispondente risolvendo la sfida.
5. La connessione viene stabilita senza che la chiave-privata abbia mai lasciato il client.

3.2.2 ssh-keygen per la firma di file

Oltre alla gestione delle chiavi *SSH*$_G$, `ssh-keygen` offre una funzionalità meno nota ma molto utile: la firma e verifica di file arbitrari. Questo è il meccanismo che *RVC*$_G$ utilizza per firmare crittograficamente i *commit*.

Il comando per firmare un file è:

```bash
ssh-keygen -Y sign -n <namespace> -f <chiave_privata> <file>
```

Il parametro `-n` specifica il *namespace*, una stringa che identifica il contesto d'uso della firma e previene attacchi di riutilizzo — una firma prodotta con *namespace*$_G$ `git` non può essere spacciata per una firma con *namespace*$_G$ `file`.

Il comando per verificare una firma è:

```bash
ssh-keygen -Y verify -n <namespace> -f <allowed_signers> -I <identità> -s <firma> < <file>
```

Il file `allowed_signers` è un elenco di identità autorizzate con le rispettive chiavi pubbliche, nel formato:

```
identita@esempio.com ssh-ed25519 AAAA...chiave...
```

Se la firma è valida e l'identità è presente nel file, il comando restituisce `Good "file" signature for <identità>`.

3.3 AGE

AGE$_G$ (*Actually Good Encryption*) è uno strumento moderno per la cifratura di file, progettato con l'obiettivo di essere semplice, sicuro e componibile. A differenza di PGP, che nel corso degli anni ha accumulato una complessità notevole, *AGE*$_G$ offre un'interfaccia minimale con poche opzioni ben definite.

AGE$_G$ supporta tre modalità di cifratura:

- **Chiave-pubblica**: il file viene cifrato con la chiave-pubblica del destinatario e può essere decifrato solo con la corrispondente chiave-privata.
- **Chiave *SSH***$_G$: *AGE*$_G$ può utilizzare le stesse chiavi *SSH*$_G$ già esistenti (incluse le chiavi *Ed25519*$_G$) come chiavi di cifratura, senza bisogno di generare nuove chiavi dedicate.
- ***Passphrase***$_G$: il file viene cifrato con una *passphrase*$_G$, usando la funzione di derivazione `scrypt` per proteggersi da attacchi a dizionario.

Un file cifrato con *AGE*$_G$ contiene nell'intestazione le informazioni necessarie per la decifratura (il tipo di chiave usata e la chiave di sessione cifrata), seguite dal contenuto

cifrato con ChaCha20-Poly1305, un algoritmo moderno che garantisce sia riservatezza che integrità.

Nel contesto di questo progetto, *AGE*$_G$ è stato studiato come tecnologia di riferimento per la fase successiva del lavoro, che prevede la progettazione di *repository* cifrate con distribuzione dei permessi di accesso. In questa fase *AGE*$_G$ permette di cifrare il contenuto di una *repository* in modo che solo gli utenti autorizzati — identificati dalle loro chiavi pubbliche — possano decifrarla, senza dover condividere nessuna chiave segreta.


```
age-encryption.org/v1
-> ssh-ed25519 Yu405A MJl34qS7ezoxXy9FQjl5TVe1BLjrz1hYEpxVxaLqFy8
bs84Hjf93u1AyipJZ50oe2NxvEeZk1EHkY1/uDZW5pI
--- RxYyA9D3tPSdSdvlcGsVOp/0oOT9XMWCJ1ffjI7mEcs
‡□□□□Í,<.'›0¹êôx9□(NÎÿ
□P§Sø†UùË□Pµ[□=tî,…¼i'»@IÒþÙ#ëÇÜtlF„ô□¼x□□|J3]Fb“°Juž{§bŽ#□Àg□é¸{Œ□ÍT–□□·□F½9*ÃÒ
```


Esempio di un file cifrato con AGE

3.4 Sistemi di controllo versione

3.4.1 Storia e evoluzione

I sistemi di controllo versione (*Version Control System*, *VCS*$_G$) nascono dalla necessità pratica di tenere traccia delle modifiche al codice sorgente nel tempo. Prima della loro diffusione, era comune affidare il versionamento a convenzioni manuali: copie di file con date nel nome, cartelle numerate, archivi compressi. Questo approccio era fragile, difficile da gestire in team e privo di qualsiasi garanzia di integrità.

Il primo sistema formale di controllo versione ampiamente adottato fu **SCCS** (*Source Code Control System*), sviluppato da Bell Labs nel 1972. SCCS introduceva il concetto di *delta*: invece di salvare ogni versione completa del file, memorizzava solo le differenze rispetto alla versione precedente, riducendo significativamente lo spazio occupato.

RCS (*Revision Control System*), sviluppato nel 1982, migliorò SCCS rendendolo più accessibile e introducendo il concetto di *branching*, ossia la possibilità di sviluppare linee di codice parallele a partire da un punto comune.

Entrambi questi sistemi operavano però a livello di singolo file e su singola macchina. Il passo successivo fu l'introduzione dei sistemi *centralizzati*.

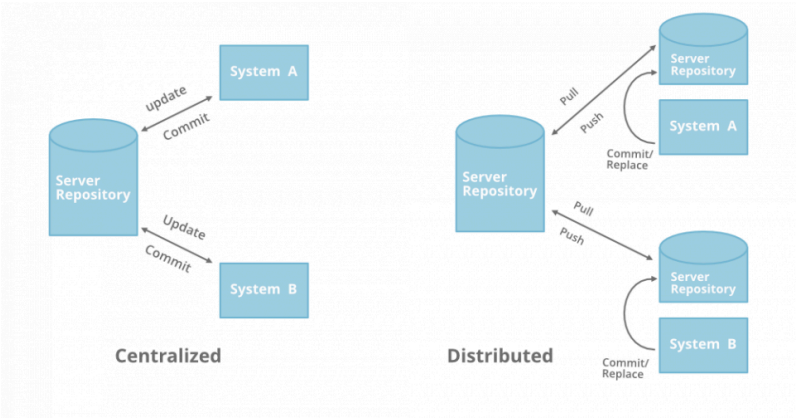
3.4.1.1 Sistemi centralizzati

CVS (*Concurrent Versions System*), negli anni "90, portò il controllo versione in rete. Per la prima volta più sviluppatori potevano lavorare contemporaneamente sullo stesso

progetto, con un server centrale che coordinava le modifiche. Il modello era semplice: il server custodisce l"intera storia del progetto; i client effettuano *checkout* (scaricano una versione) e *commit* (inviano le modifiche).

Subversion (SVN), rilasciato nel 2000, nacque esplicitamente come sostituto migliorato di CVS, correggendo molte delle sue limitazioni tecniche. SVN trattava l'intera struttura del progetto come un'unità atomica: un *commit*$_G$ poteva riguardare più file contemporaneamente, con la garanzia che o tutte le modifiche venivano salvate o nessuna.

Il limite fondamentale dei sistemi centralizzati era però strutturale: la presenza di un singolo punto di fallimento. Se il server non era raggiungibile, nessuno poteva fare *commit*$_G$. Se il server veniva perso, si perdeva l'intera storia del progetto.



Controllo versioni centralizzato vs distribuito

3.4.1.2 Sistemi distribuiti

Git$_G$, sviluppato da Linus Torvalds nel 2005 per gestire lo sviluppo del kernel Linux, rivoluzionò il campo introducendo un modello completamente distribuito. In *Git*$_G$ non esiste un server centrale: ogni sviluppatore possiede una copia completa dell'intera storia del progetto. I *commit*$_G$ avvengono localmente e possono essere sincronizzati con altri *repository*$_G$ in un secondo momento.

Questa architettura offre vantaggi significativi: si può lavorare offline, la storia del progetto è replicata su ogni macchina riducendo il rischio di perdita dei dati, e il *branching* è diventato un'operazione economica e centrale nel flusso di lavoro.

Mercurial, rilasciato nello stesso anno di *Git*$_G$, adottò un approccio simile ma con un'interfaccia considerata più accessibile. Oggi *Git*$_G$ domina il mercato, ma l'ecosistema dei *VCS*$_G$ distribuiti rimane vivo con strumenti come Fossil e Pijul.

3.4.2 Git in dettaglio

In *Git*$_G$ ogni oggetto — *blob* (contenuto di un file), *tree* (struttura di una directory), *commit*, *tag* — è identificato da un *hash*$_G$ SHA del suo contenuto. Questo significa che l'identità di ogni oggetto è determinata dal suo contenuto: due oggetti con lo stesso contenuto hanno lo stesso *hash*$_G$, e qualsiasi modifica produce un *hash*$_G$ diverso.

Un *commit* *Git*$_G$ contiene: il riferimento all'albero dei file in quello stato, il riferimento al *commit*$_G$ precedente (*parent*), i metadati dell'autore e del committer, il messaggio di *commit*$_G$. Questa struttura crea una catena crittograficamente collegata: modificare un *commit*$_G$ invalida tutti i *commit*$_G$ successivi, poiché i loro *hash*$_G$ dipendono dal *hash*$_G$ del precedente.

Git$_G$ supporta la firma crittografica dei *commit*$_G$ tramite GPG o *SSH*$_G$. Tuttavia questa funzionalità è opzionale e deve essere abilitata esplicitamente — non fa parte del flusso di lavoro standard.

3.4.3 RVC a confronto con Git

RVC$_G$ condivide con *Git*$_G$ il modello distribuito ma si differenzia in aspetti fondamentali di architettura e sicurezza.

Caratteristica	***Git*$_G$**	***RVC*$_G$**
Struttura	*Repository* con oggetti indicizzati	File ZIP navigabili su file-system
Identificazione *commit*$_G$	*Hash*$_G$ SHA dell'oggetto *commit*$_G$	Timestamp codificato in base36
Firma *commit*$_G$	Opzionale (GPG o *SSH*$_G$)	Integrata nell'architettura
Server centrale	Non richiesto ma comune	Non richiesto per design
Repository$_G$ multiple	Un remote alla volta tipicamente	Più *repository*$_G$ sincronizzate nativamente
Linguaggio	C	*CPL*$_G$

Confronto tra Git e RVC.

La differenza più significativa riguarda la sicurezza: mentre in *Git*$_G$ la firma è un'opzione che il singolo sviluppatore può scegliere di abilitare o meno, in *RVC*$_G$ è parte del modello

stesso. Ogni *commit* produce un file `.sig` che contiene gli *hash*$_G$ crittografici del contenuto e della catena precedente, costruendo una struttura analoga a una *blockchain*: modificare un *commit*$_G$ invalida tutti quelli successivi perché l'hash cumulativo non corrisponde più.

3.5 RVC: architettura e funzionamento

3.5.1 Struttura della repository

Una *repository* *RVC*$_G$ è una semplice cartella sul filesystem, senza strutture dati complesse o indici da mantenere. Ogni *commit* è rappresentato da due file:

- Un archivio **ZIP** contenente il *commit* del progetto nella versione corrispondente, incluso il file `.FileManifest` che descrive lo stato di tutti i file tracciati.
- Un file **.sig** contenente i metadati del *commit*$_G$ e, opzionalmente, la firma *SSH*$_G$.

I file seguono una convenzione di denominazione che codifica la struttura della storia:

```
1 <progetto>.<id>.<idPrecedente>.{autore}+tag.zip
2 <progetto>.<id>.sig
```

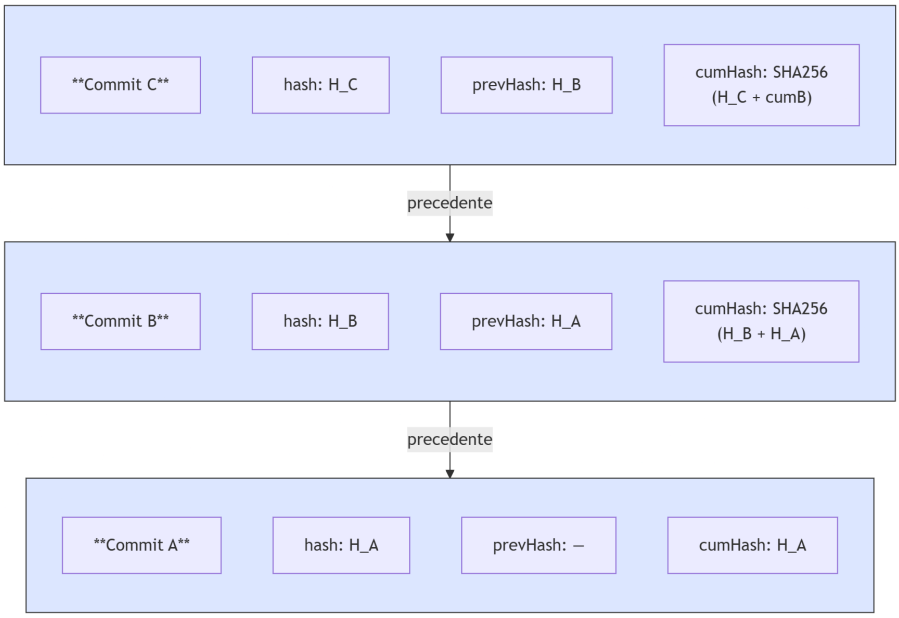
L'`id` è un timestamp codificato in base36 (cifre 0-9 e lettere A-Z), che permette l'ordinamento cronologico dei *commit*$_G$ semplicemente confrontando i nomi dei file. Il riferimento al *commit*$_G$ precedente è incorporato nel nome del file ZIP, rendendo la struttura della storia navigabile senza alcun indice aggiuntivo.

3.5.2 Il file .sig e la blockchain degli hash

Il file `.sig` è il cuore del sistema di sicurezza di *RVC*$_G$. Contiene in formato binario proprietario i seguenti campi:

- `author`: il nome dell'autore del *commit*$_G$
- `comment`: il messaggio del *commit*$_G$
- `fn`: il nome del file ZIP corrispondente
- `id`: l'identificativo del *commit*$_G$
- `prevId`: l'identificativo del *commit*$_G$ precedente
- `hash`: lo SHA256 del file ZIP di questo *commit*$_G$
- `prevHash`: lo SHA256 del file ZIP del *commit*$_G$ precedente
- `cumulativeHash`: lo SHA256 della concatenazione dell'hash attuale con il `cumulativeHash` del *commit*$_G$ precedente

Il `cumulativeHash` è la chiave della sicurezza: ogni *commit*$_G$ incorpora crittograficamente l'intera storia precedente. Verificare che il `cumulativeHash` di un *commit*$_G$ sia corretto significa verificare implicitamente che tutti i *commit*$_G$ precedenti siano integri.



Struttura del file .sig e la catena degli hash cumulativi

Dopo i metadati, il file `.sig` contiene una firma *SSH*$_G$ nel formato standard:

```
1 -----BEGIN SSH SIGNATURE-----
2 ...
3 -----END SSH SIGNATURE-----
```

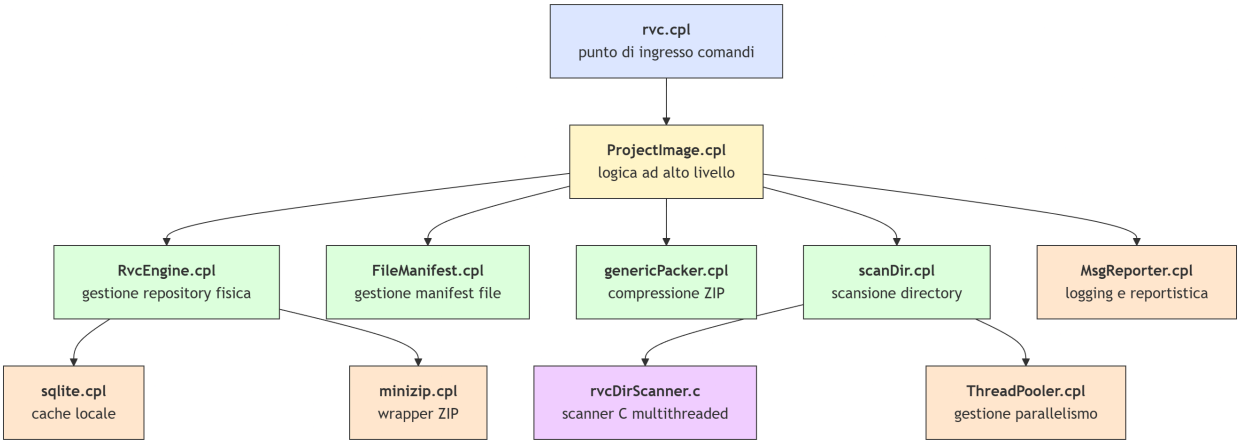
Questa firma attesta che l'autore dichiarato ha effettivamente prodotto il *commit*$_G$, rendendo ogni modifica crittograficamente attribuibile.

3.5.3 Il linguaggio CPL

RVC$_G$ è scritto in ***CPL***$_G$ (*CodePainter Language*), un linguaggio proprietario sviluppato da Zucchetti S.p.A. *CPL*$_G$ è un linguaggio interpretato tipizzato staticamente, con supporto a classi, moduli e gestione dei file. Viene eseguito tramite un interprete (`cpl.exe`) che supporta sia interpretazione diretta che compilazione *JIT*$_G$.

Le caratteristiche principali che distinguono *CPL*$_G$ dai linguaggi comuni includono la sintassi di assegnazione con `:=`, l'assenza dell'istruzione `return` esplicita (si usa invece la variabile implicita `result`), la dichiarazione obbligatoria di tutte le variabili in cima alla funzione prima di qualsiasi blocco di codice, e la distinzione tra `func` (funzione con valore di ritorno) e `proc` (procedura senza valore di ritorno).

Il codice sorgente di *RVC*$_G$ è organizzato in diversi moduli *CPL*$_G$, ciascuno con responsabilità ben definite: `ProjectImage.cpl` contiene la logica ad alto livello, `RvcEngine.cpl` gestisce la *repository*$_G$ fisica, `FileManifest.cpl` gestisce il *manifest*$_G$ dei file tracciati.



Struttura dei moduli principali di RVC

3.5.4 Flusso di un commit

Quando un utente esegue `rvc commit`, il sistema esegue i seguenti passi:

1. Legge il file `.FileManifest` nella directory di lavoro per conoscere lo stato corrente del progetto.
2. Scansiona la directory e calcola le differenze rispetto allo stato precedente.
3. Crea un archivio ZIP con i file modificati e il nuovo `.FileManifest`.
4. Calcola lo SHA256 dell'archivio ZIP.
5. Recupera l'hash e il `cumulativeHash` del *commit*$_G$ precedente dal suo file `.sig`.
6. Calcola il nuovo `cumulativeHash` come SHA256 della concatenazione dell'hash attuale con il `cumulativeHash` precedente.
7. Crea il file `.sig` con tutti i metadati.
8. Esegue `ssh-keygen -Y sign` per firmare il file `.sig` e accoda la firma al file.
9. Copia i due file nella *repository*$_G$.

Capitolo 4

Requisiti e modello di sicurezza

In questo capitolo viene definito il modello di sicurezza che un sistema di versionamento distribuito moderno dovrebbe soddisfare. Partendo dalle proprietà fondamentali richieste, vengono analizzate le scelte architetturali, i trade-off e le motivazioni che portano alla definizione di un sistema gerarchico di fiducia, di livelli di sicurezza configurabili e di requisiti formali. Il capitolo si conclude con un analisi del divario che confronta il modello ideale con lo stato attuale di RVC, identificando le aree di intervento affrontate nei capitoli successivi.

4.1 Sicurezza strutturale nei sistemi di versionamento distribuito

Un sistema di versionamento distribuito è considerato sicuro quando garantisce, per ogni operazione e indipendentemente dal canale di distribuzione, le seguenti proprietà fondamentali.

- **Integrità** — il contenuto di ogni *commit*$_G$ non può essere alterato senza che la modifica sia rilevabile. Questa proprietà è garantita da funzioni di *hash*$_G$ crittografiche: dato un archivio ZIP, la sua impronta *SHA-256*$_G$ è univoca e deterministica. Qualsiasi modifica — anche di un singolo byte — produce un hash completamente diverso. Un sistema che garantisce l'integrità permette a chiunque di verificare che il contenuto ricevuto sia identico a quello prodotto dall'autore, senza dover contattare nessuna fonte autoritativa.
- **Autenticità** — ogni *commit*$_G$ è crittograficamente attribuibile all'autore che l'ha prodotta. L'autenticità è garantita dalla *firma digitale*$_G$: l'autore firma il *commit*$_G$ con la propria *chiave privata*$_G$ e chiunque può verificare la firma usando la corrispondente *chiave pubblica*$_G$. Senza autenticità, un sistema può garantire che i dati non siano stati modificati durante il trasferimento, ma non può garantire chi li abbia prodotti originariamente.
- **Non ripudio** — un autore non può negare di aver prodotto una *commit*$_G$ firmata con la propria *chiave privata*$_G$. Il non ripudio è una conseguenza diretta della firma digitale: poiché solo il possessore della chiave privata può produrre una firma valida, la presenza di una firma valida è prova crittografica della paternità. Questa proprietà

è rilevante in contesti contrattuali e legali — se un fornitore di software distribuisce codice firmato, non può successivamente sostenere di non averlo prodotto.

- **Ordine verificabile** — la sequenza temporale delle modifiche è verificabile e non manipolabile retroattivamente. Questa proprietà è la più complessa da garantire in un sistema distribuito. Non è sufficiente che ogni *commit*$_G$ sia integra e autentica — è necessario anche che la loro sequenza sia crittograficamente vincolata, in modo che inserire, rimuovere o riordinare commit sia rilevabile. Questa proprietà è garantita dalla struttura a catena degli hash: ogni *commit*$_G$ incorpora l'hash della precedente, rendendo impossibile modificare un elemento della catena senza invalidare tutti quelli successivi.

4.1.1 Confronto con i sistemi esistenti

In *Git*$_G$ l'integrità è garantita dalla catena di hash *SHA-256*$_G$ — modificare una *commit*$_G$ invalida tutte quelle successive. Autenticità e non ripudio sono invece opzionali: la firma crittografica tramite *SSH*$_G$ o GPG deve essere abilitata esplicitamente e non fa parte del flusso di lavoro standard. L'ordine temporale non è verificabile in senso assoluto — *Git*$_G$ non verifica i timestamp e permette la creazione di *commit*$_G$ con date arbitrarie:

```bash
GIT_AUTHOR_DATE="2020-01-01T00:00:00" git commit -m "commit retrodatata"
```

Il risultato è una *commit*$_G$ inserita nella storia con una data arbitraria, indistinguibile da una *commit*$_G$ legittima. Questa non è una vulnerabilità ma una scelta progettuale documentata — *Git*$_G$ garantisce l'ordine relativo tramite la catena di hash, non l'ordine temporale assoluto.

La distinzione tra disponibilità e correttezza di un sistema centralizzato è emersa concretamente il 23 aprile 2026, quando un bug nella funzionalità di merge queue di GitHub ha causato la generazione silenziosa di *commit*$_G$ errate per 2.092 pull request in 658 *repository*$_G$. Le modifiche precedentemente unite sono state ripristinate dai merge successivi senza alcun avviso. La piattaforma era tecnicamente operativa durante tutto l'incidente — gli sviluppatori potevano ancora fare push, aprire pull request e fare merge. Il fatto che il merge stesse corrompendo silenziosamente il codice non risultava come incidente nella dashboard di stato. Disponibilità e correttezza non sono la stessa proprietà: un sistema può essere raggiungibile e simultaneamente produrre risultati errati. Un sistema distribuito che incorpora le garanzie crittografiche descritte sopra non dipende dalla correttezza del server per verificare l'integrità dei propri dati.

4.1.2 Applicazione in RVC

RVC$_G$ garantisce l'integrità tramite il `cumulativeHash` — ogni *commit*$_G$ incorpora crittograficamente l'intera storia precedente. La firma *SSH*$_G$ garantisce autenticità e non ripudio. L'ordine temporale presenta la stessa limitazione di *Git*$_G$ — gli identificativi sono timestamp codificati in base36 basati sull'orologio della macchina. Il `cumulativeHash` garantisce però l'ordine **relativo**: anche con timestamp errati o manipolati, la sequenza crittografica è verificabile e non manipolabile senza invalidare l'intera catena. Questa limitazione viene documentata come scelta consapevole — per il contesto d'uso di *RVC*$_G$ la garanzia di ordine relativo è sufficiente e la complessità aggiuntiva di un sistema di timestamping certificato non è giustificata.

Per mitigare parzialmente questa limitazione, il modello proposto prevede che l'identificativo di ogni *commit*$_G$ sia composto da due componenti: il timestamp codificato in base36 per mantenere l'ordinamento cronologico visivo, e una porzione dell'hash del contenuto per garantire l'unicità crittografica. Un identificativo nella forma `0Q6JTD7XVZ_A3F2B1C4` — dove la prima parte è il timestamp e la seconda sono i primi otto caratteri dell'*hash*$_G$ *SHA-256*$_G$ dell'archivio ZIP — rende praticamente impossibile la collisione intenzionale mantenendo la leggibilità e l'ordinamento per nome file. Questa modifica è identificata come requisito nel modello ideale e discussa nella gap analysis nella Sezione 4.7.

4.2 Requisiti di sicurezza formali

A partire dalle proprietà definite nella sezione precedente, è possibile derivare un insieme di requisiti formali che un sistema di versionamento distribuito sicuro deve soddisfare. Ogni requisito è classificato per priorità obbligatorio (O) o desiderabile (D) e verrà ripreso nella *gap analysis* per valutare lo stato attuale di *RVC*$_G$.

4.2.1 Integrità e ordine verificabile

L'integrità è la proprietà fondamentale di qualsiasi sistema di versionamento — senza di essa non è possibile fidarsi del contenuto ricevuto. In un sistema distribuito questa garanzia non può dipendere dalla fiducia nel canale di distribuzione ma deve essere incorporata nei dati stessi. Ogni *commit*$_G$ deve portare con sé la prova crittografica della propria integrità, e la struttura della catena deve rendere rilevabile qualsiasi tentativo di modifica retroattiva. Il problema dell'ordine temporale assoluto, come discusso nella sezione precedente, viene trattato come limitazione documentata piuttosto che come requisito da risolvere completamente — il sistema deve però garantire l'ordine relativo e rendere non manipolabili gli identificativi dei *commit*$_G$.

Codice	Descrizione	Priorità
RS01	Ogni *commit*$_G$ deve essere verificabile tramite *hash*$_G$ crittografico del proprio contenuto	O
RS02	La catena degli hash deve essere verificabile in modo che qualsiasi modifica a una *commit*$_G$ invalidi tutte le successive	O
RS03	Gli identificativi dei *commit*$_G$ devono essere univoci e non manipolabili tramite timestamp arbitrari	O
RS04	Il sistema deve documentare esplicitamente le proprie limitazioni in termini di ordine temporale assoluto	O

Requisiti di integrità e ordine verificabile.

4.2.2 Autenticità e non ripudio

Garantire l'integrità del contenuto non è sufficiente se non è possibile stabilire chi lo ha prodotto. L'autenticità richiede che ogni *commit*$_G$ sia crittograficamente attribuibile al suo autore tramite firma digitale. Il non ripudio è una conseguenza diretta di questa scelta — un autore non può negare di aver prodotto una *commit*$_G$ firmata con la propria *chiave privata*$_G$. La radice di fiducia dell'intera *repository*$_G$ è la prima *commit*$_G$, che deve essere verificabile autonomamente da qualsiasi terzo in possesso della chiave pubblica dell'amministratore, senza dipendere da infrastrutture esterne.

Codice	Descrizione	Priorità
RS05	La prima *commit*$_G$ di ogni *repository*$_G$ deve costituire una radice di fiducia verificabile autonomamente	O
RS06	Il sistema deve supportare e imporre la firma digitale tramite chiave *SSH*$_G$ per ogni *commit*$_G$ nei progetti configurati con livello di sicurezza maggiore o uguale a 1	O

Requisiti di autenticità e non ripudio.

4.2.3 Gestione delle identità

In un sistema multi-utente la gestione delle identità è il meccanismo che traduce le garanzie crittografiche in un modello organizzativo concreto. Non è sufficiente che le firme

siano tecnicamente valide — è necessario che il sistema definisca chi è autorizzato a firmare, come vengono gestiti i cambi di personale e chi ha il potere di modificare questi elenchi. La gerarchia a tre livelli — amministratore, responsabile e dipendente — riflette la struttura organizzativa tipica di un'azienda software e permette di delegare la gestione dei permessi mantenendo un controllo centralizzato sulla radice di fiducia. La revoca deve essere immediata e non richiedere operazioni straordinarie — è sufficiente aggiornare il file di autorizzazione nella *commit*$_G$ successiva.

Codice	Descrizione	Priorità
RS07	Il sistema deve supportare una gerarchia di fiducia a tre livelli: amministratore, responsabile e dipendente	O
RS08	I permessi di scrittura devono essere configurabili per progetto tramite un file di autorizzazione versionato	O
RS09	La revoca di un'identità deve essere efficace dalla *commit*$_G$ successiva alla modifica del file di autorizzazione	O
RS10	La successione di un responsabile deve essere gestita esclusivamente dall'amministratore del sistema	D

Requisiti di gestione delle identità.

4.2.4 Sicurezza configurabile

Non tutti i progetti richiedono lo stesso livello di protezione. Un prototipo interno ha esigenze diverse da un modulo che gestisce dati sensibili di un cliente. Imporre lo stesso livello di sicurezza a tutti i progetti sarebbe eccessivamente restrittivo per alcuni e insufficiente per altri. Il modello proposto prevede livelli di sicurezza configurabili per progetto, con il vincolo che il livello non possa essere abbassato nel tempo — una scelta che previene attacchi che cercano di degradare le garanzie di sicurezza di un progetto già avviato. Per i progetti che richiedono la massima riservatezza, il contenuto delle *commit*$_G$ può essere cifrato con *AGE*$_G$, rendendo il codice leggibile solo agli utenti autorizzati. La gestione della visibilità dei destinatari introduce un ulteriore grado di configurabilità, permettendo di bilanciare usabilità e privacy in base al contesto.

Codice	Descrizione	Priorità
RS11	Il sistema deve supportare livelli di sicurezza configurabili per progetto, non abbassabili nel tempo	D
RS12	Il contenuto delle *commit*$_G$ deve poter essere cifrato con *AGE*$_G$ per progetti riservati	D
RS13	La gestione dei destinatari nei progetti cifrati deve supportare modalità configurabili di visibilità	D

Requisiti di sicurezza configurabile.

4.2.5 Gestione dei branch

I branch sono uno strumento fondamentale nello sviluppo software parallelo, ma introducono scenari di sicurezza che vanno gestiti esplicitamente. Un branch può diventare inutile al termine di una funzionalità, oppure può risultare compromesso a seguito di una *commit*$_G$ fraudolenta o non autorizzata. In entrambi i casi il sistema deve fornire un meccanismo formale per dichiarare lo stato del branch, senza cancellare la storia — che rimane immutabile e verificabile — ma aggiungendo una *commit*$_G$ firmata che ne attesti la chiusura o la compromissione. Questo approccio mantiene la tracciabilità completa degli eventi, inclusa la prova della compromissione stessa.

Codice	Descrizione	Priorità
RS14	I branch compromessi devono poter essere chiusi con una *commit*$_G$ firmata che ne attesti la compromissione	D

Requisiti di gestione dei branch.

I requisiti obbligatori definiscono le proprietà minime senza le quali il sistema non può essere considerato sicuro per il contesto d'uso descritto. I requisiti desiderabili estendono il modello con funzionalità che aumentano significativamente il livello di sicurezza, ma la cui assenza non compromette le garanzie fondamentali.

I requisiti RS01, RS02 e RS03 corrispondono alla proprietà di integrità e alla gestione dell'ordine verificabile. RS05 e RS06 garantiscono le proprietà di autenticità e non ripudio. RS07, RS08, RS09 e RS10 definiscono il modello di gestione delle identità e dei permessi. RS11, RS12 e RS13 estendono il modello con funzionalità di sicurezza configurabile. RS04

e RS14 affrontano rispettivamente la documentazione delle limitazioni note e la gestione degli incidenti sui branch.

4.3 Gerarchia di fiducia

In un sistema di versionamento distribuito la fiducia non può essere delegata a un server centrale — deve essere incorporata nella struttura stessa dei dati. Il modello proposto definisce una gerarchia a tre livelli operativi più un livello esterno di sola lettura, ciascuno con responsabilità e permessi ben definiti. La gerarchia è asimmetrica: ogni livello superiore può esercitare i poteri del livello inferiore, ma non viceversa.

4.3.1 Amministratore (CapoProgetto)

L'amministratore è la radice assoluta di fiducia dell'intera *repository*$_G$. Dispone di due chiavi crittografiche distinte: una chiave operativa utilizzata nelle operazioni quotidiane e una chiave master conservata offline su un dispositivo air-gapped o in una cassaforte fisica. La separazione tra le due chiavi limita la finestra di rischio in caso di compromissione: se la chiave operativa viene rubata o esposta, l'amministratore usa la chiave master per revocarla e nominarne una nuova senza perdere il controllo della *repository*$_G$.

La prima operazione alla creazione di una *repository*$_G$ è produrre il file `allowed_Responsabili` — l'elenco delle chiavi pubbliche dei responsabili autorizzati — e firmarlo con la chiave privata operativa. Questo file e la sua firma costituiscono la prima *commit*$_G$ della *repository*$_G$ e la radice di fiducia da cui deriva tutta la catena di verifica successiva. L'amministratore ha accesso in lettura e scrittura a tutti i progetti della *repository*$_G$ a qualsiasi livello di sicurezza, incluso il livello 4 con contenuto cifrato — la sua chiave pubblica è sempre inclusa tra i destinatari autorizzati.

4.3.2 Responsabile di progetto

Il responsabile gestisce uno o più progetti all'interno della *repository*$_G$. Il ruolo viene assegnato dall'amministratore tramite inclusione nel file `allowed_Responsabili` — non può essere auto-assegnato né delegato a un altro responsabile. Per ogni progetto gestito, il responsabile mantiene il file `allowed_Dipendenti`, che elenca le chiavi pubbliche dei dipendenti autorizzati a committare. Questo file è versionato all'interno dello ZIP di ogni *commit*$_G$ del progetto, fa parte del contenuto hashato e firmato, e la sua storia è completamente tracciabile.

Il responsabile aggiunge o rimuove dipendenti dal proprio progetto in autonomia. L'amministratore può sovrascrivere il file `allowed_Dipendenti` di qualsiasi progetto in qualsiasi momento — il suo potere non è vincolato dalla struttura gerarchica. Se

un responsabile lascia l'azienda o viene rimosso dal ruolo, l'amministratore nomina un sostituto aggiornando il file `allowed_Responsabili`. Fino alla nomina del sostituto il progetto entra in stato di attesa: i dipendenti esistenti possono continuare a committare, ma non è possibile aggiungere nuovi dipendenti né modificare i permessi esistenti.

4.3.3 Dipendente

Il dipendente è autorizzato a committare su un progetto se e solo se la propria chiave pubblica è presente nel file `allowed_Dipendenti` della *commit*$_G$ più recente di quel progetto. I permessi sono definiti per progetto — un dipendente autorizzato su ProgettoA non ha accesso a ProgettoB, anche se entrambi appartengono alla stessa *repository*$_G$. Non esistono permessi per branch: un dipendente autorizzato su un progetto può committare su qualsiasi branch di quel progetto.

La revoca è operativa dalla *commit*$_G$ successiva alla modifica del file `allowed_Dipendenti`: il dipendente rimosso non può produrre *commit*$_G$ valide sul progetto. Le *commit*$_G$ prodotte prima della revoca rimangono valide in quanto firmate da un'identità che era autorizzata al momento della firma — la storia del progetto è immutabile e ogni modifica ai permessi è tracciata nella catena.

4.3.4 Cliente

Il cliente riceve la *repository*$_G$ e verifica autonomamente autenticità e integrità del contenuto, senza dipendere dall'infrastruttura del produttore. La verifica parte dalla chiave pubblica operativa dell'amministratore, ottenuta tramite un canale indipendente dalla *repository*$_G$ stessa — ad esempio il sito ufficiale del produttore distribuito tramite HTTPS. Con questa chiave il cliente verifica la firma sul file `allowed_Responsabili` della prima *commit*$_G$ e da lì risale crittograficamente all'intera catena di autorizzazioni e *commit*$_G$.

Il cliente non ha permessi di scrittura sulla *repository*$_G$. Quando riceve un aggiornamento, verifica che le nuove *commit*$_G$ si colleghino correttamente alla catena già in suo possesso, partendo dalla stessa radice di fiducia. Nei progetti di livello 4 la chiave pubblica del cliente deve essere registrata tra i destinatari autorizzati per poter decifrare il contenuto — la verifica della catena e delle firme è comunque possibile senza decifrare, poiché l'hash nel file `.sig` è calcolato sul contenuto cifrato.

4.3.5 Inizializzazione della repository

La creazione di una nuova *repository*$_G$ segue questa procedura:

1. L'amministratore genera la coppia di chiavi master con `ssh-keygen -t ed25519` e conserva la chiave privata master su un dispositivo offline. La chiave pubblica master è l'ancora di fiducia primaria del sistema.
2. L'amministratore genera la coppia di chiavi operativa con `ssh-keygen -t ed25519`. La chiave pubblica operativa viene firmata con la chiave privata master, creando un certificato che attesta la delega operativa.
3. L'amministratore crea il file `allowed_Responsabili` con le chiavi pubbliche dei responsabili nominati e lo firma con la chiave privata operativa.
4. Il file `allowed_Responsabili`, la sua firma e il certificato della chiave operativa costituiscono la prima *commit*$_G$ della *repository*$_G$ — la radice di fiducia verificabile da qualsiasi terzo in possesso della chiave pubblica master.
5. La chiave pubblica master viene pubblicata su un canale indipendente dalla *repository*$_G$. Chiunque la possieda può verificare la legittimità della chiave operativa e, da lì, tutta la catena di autorizzazioni.

In caso di compromissione della chiave operativa, l'amministratore genera una nuova coppia operativa, la firma con la chiave master e pubblica una *commit*$_G$ di revoca che dichiara la vecchia chiave non più valida e introduce la nuova. La chiave master non viene mai esposta durante le operazioni ordinarie — il suo utilizzo è limitato alla firma della chiave operativa e alle operazioni di revoca.

4.3.6 Inizializzazione di un progetto

La creazione di un nuovo progetto all'interno di una *repository*$_G$ esistente è gestita dal responsabile nominato per quel progetto, senza necessità di intervento dell'amministratore:

1. Il responsabile crea il file `allowed_Dipendenti` con le chiavi pubbliche dei dipendenti autorizzati.
2. Il responsabile crea il file `.rvc_policy` con il livello di sicurezza scelto per il progetto.
3. Il responsabile produce la prima *commit*$_G$ del progetto, firmata con la propria chiave privata. La firma è verificabile tramite il file `allowed_Responsabili` della *repository*$_G$.
4. Da questo momento i dipendenti elencati in `allowed_Dipendenti` sono autorizzati a committare sul progetto.

Il livello di sicurezza definito nella prima *commit*$_G$ non può essere abbassato — può essere alzato in qualsiasi momento tramite una *commit*$_G$ firmata dal responsabile o dall'amministratore. Questa scelta elimina la possibilità di degradare le garanzie di sicurezza di un progetto già avviato.

4.3.7 File amministrativi della repository

In RVC ogni *commit*$_G$ appartiene a un progetto — non esiste il concetto di *commit*$_G$ globale della *repository*$_G$. I file amministrativi `allowed_Responsabili` e la sua firma devono però risiedere nella *repository*$_G$ in modo verificabile e versionato, indipendentemente da qualsiasi progetto specifico.

Il modello proposto risolve questo problema definendo un progetto riservato con nome convenzionale `_rvc_root`, dedicato esclusivamente all'amministrazione della *repository*$_G$. Solo l'amministratore è autorizzato a committare su questo progetto — il file `allowed_Responsabili` e la sua firma risiedono all'interno dello ZIP di ogni *commit*$_G$ di `_rvc_root`, seguendo la stessa struttura di qualsiasi altro progetto. La verifica della radice di fiducia usa lo stesso codice che verifica qualsiasi altra *commit*$_G$ — nessun caso speciale è necessario nel motore.

Il nome `_rvc_root` è riservato per convenzione del modello. Per prevenire conflitti, il motore verifica all'inizializzazione che questo nome non sia già in uso e lo riserva automaticamente — qualsiasi tentativo di creare un progetto con questo nome da parte di un responsabile o dipendente viene rifiutato. Questa è l'unica modifica richiesta al motore di RVC rispetto al comportamento standard.

Questa scelta è preferita all'alternativa di file speciali nella radice della *repository*$_G$ perché non richiede modifiche architetturali al motore e mantiene la coerenza del modello — la verifica della radice di fiducia usa esattamente la stessa logica della verifica di qualsiasi altro progetto.

4.3.8 Catena di fiducia tra progetti

Il progetto `_rvc_root` non collega crittograficamente i progetti della *repository*$_G$ tra loro — ogni progetto mantiene una propria catena di *commit*$_G$ indipendente. La funzione di `_rvc_root` è certificare le identità autorizzate, non la struttura dei dati. La fiducia tra i progetti è gerarchica attraverso le identità, non crittografica attraverso la struttura.

La verifica di una *commit*$_G$ di un qualsiasi progetto segue questa catena:

1. La firma della *commit*$_G$ viene verificata contro le chiavi presenti in `allowed_Dipendenti`.
2. La firma di `allowed_Dipendenti` viene verificata contro la chiave del responsabile.
3. La chiave del responsabile viene verificata contro `allowed_Responsabili` presente in `_rvc_root`.
4. La firma di `allowed_Responsabili` viene verificata con la chiave pubblica master dell'amministratore, ottenuta fuori banda.

Questa catena implica un requisito operativo: la verifica completa di qualsiasi *commit*$_G$ richiede la presenza di `_rvc_root` nella *repository*$_G$. Chi riceve la *repository*$_G$ riceve automaticamente tutti i progetti incluso `_rvc_root` — ma un sistema che distribuisce solo i file di un singolo progetto non permette la verifica completa della catena di fiducia.

4.3.9 Implicazioni di sicurezza della radice pubblica

Il progetto `_rvc_root` non può essere cifrato — deve essere leggibile da qualsiasi soggetto che voglia verificare la catena di fiducia, incluso il cliente. Questa necessità introduce una tensione strutturale tra verificabilità pubblica e riservatezza organizzativa.

Il contenuto di `_rvc_root` espone le chiavi pubbliche dei responsabili e, implicitamente, la struttura organizzativa dell'azienda. Le chiavi pubbliche non sono segrete per definizione, ma la lista dei responsabili è informazione sensibile — rivela chi ha potere decisionale sulla *repository*$_G$ e rende questi soggetti bersagli privilegiati per attacchi di ingegneria sociale e spear phishing. Analogamente, i nomi dei file ZIP nella *repository*$_G$ rivelano i nomi dei progetti anche quando il contenuto è cifrato a livello 4.

Il modello propone due mitigazioni parziali. La prima riguarda le identità: invece di utilizzare indirizzi email o nomi reali nel file `allowed_Responsabili`, si possono adottare identificativi opachi — ad esempio `resp-001` — riducendo la leggibilità immediata senza eliminare la tracciabilità per chi ha accesso alle informazioni di mappatura. La seconda riguarda i nomi dei progetti: l'uso di identificativi non descrittivi — ad esempio `PRJ-4A2F` invece di `ModuloPagamenti` — impedisce la mappatura immediata del contenuto della *repository*$_G$ a partire dalla struttura dei file.

Queste mitigazioni riducono la superficie di esposizione ma non la eliminano. Il trade-off tra verificabilità pubblica e riservatezza organizzativa è una limitazione strutturale del modello — qualsiasi sistema che permette la verifica autonoma della catena di fiducia deve necessariamente rendere pubblica almeno la radice di quella catena.

4.4 Livelli di sicurezza configurabili

Un sistema di versionamento distribuito utilizzato in contesti aziendali deve servire esigenze di sicurezza eterogenee. Un prototipo interno in fase esplorativa, un modulo di produzione distribuito a clienti e un componente che gestisce dati finanziari hanno requisiti di protezione radicalmente diversi. Imporre un livello di sicurezza uniforme a tutti i progetti è eccessivamente restrittivo per alcuni e insufficiente per altri.

Il modello proposto introduce livelli di sicurezza configurabili per progetto, definiti alla creazione del progetto nel file `.rvc_policy` e non abbassabili nel tempo. Il vincolo di non abbassabilità è una scelta deliberata: un attaccante che compromette l'account di un

responsabile non può degradare le garanzie di sicurezza di un progetto già avviato — può solo alzarle. Ogni livello è un sovrainsieme del precedente: un progetto a livello 3 soddisfa tutti i requisiti dei livelli 0, 1 e 2.

4.4.1 Livello 0 — Aperto

Nessuna firma è richiesta. Chiunque abbia accesso fisico alla *repository*$_G$ può aggiungere *commit*$_G$. Non vengono verificate né l'identità dell'autore né l'integrità della catena. Questo livello è appropriato per prototipi interni in fase esplorativa dove la velocità di sviluppo è prioritaria e la tracciabilità formale non è richiesta. Non fornisce nessuna delle quattro proprietà di sicurezza definite nella sezione 4.1.

4.4.2 Livello 1 — Autenticato

Ogni *commit*$_G$ deve essere firmata digitalmente, ma non esiste una lista di identità autorizzate — qualsiasi chiave *SSH*$_G$ valida è accettata. Il sistema verifica che la firma sia crittograficamente valida e che corrisponda a una chiave privata reale, ma non verifica se quella chiave appartenga a un soggetto autorizzato. Questo livello garantisce autenticità e non ripudio — ogni *commit*$_G$ è attribuibile a chi possiede la chiave corrispondente — ma non autorizzazione. È appropriato per repository interne dove tutti i partecipanti sono implicitamente fidati ma si vuole mantenere la tracciabilità delle modifiche.

4.4.3 Livello 2 — Autorizzato

Ogni *commit*$_G$ deve essere firmata da una chiave presente nel file `allowed_Dipendenti` del progetto. Il sistema verifica sia la validità crittografica della firma sia l'appartenenza dell'identità alla lista degli autorizzati. Questo livello garantisce autenticità, non ripudio e autorizzazione — solo i soggetti esplicitamente nominati dal responsabile possono produrre *commit*$_G$ valide. È il livello base raccomandato per qualsiasi progetto in produzione.

4.4.4 Livello 3 — Verificato

Come il livello 2, con l'aggiunta della verifica obbligatoria della catena degli hash a ogni operazione. Nessuna operazione — lettura, aggiornamento, push — può procedere se la catena risulta corrotta o incompleta. Mentre nei livelli precedenti la verifica della catena è un'operazione esplicita eseguita su richiesta, al livello 3 è una precondizione implicita di qualsiasi interazione con il progetto. Questo livello è appropriato per codice distribuito a clienti esterni, dove la garanzia di integrità deve essere continua e non delegabile a verifiche periodiche manuali.

4.4.5 Livello 4 — Riservato

Come il livello 3, con l'aggiunta della cifratura del contenuto degli archivi ZIP tramite *AGE*$_G$. Solo i soggetti la cui chiave pubblica è registrata tra i destinatari autorizzati possono decifrare e leggere il contenuto. La verifica della catena e delle firme rimane possibile senza decifrare — l'*hash*$_G$ nel file `.sig` è calcolato sul contenuto cifrato, non su quello in chiaro. Questo livello è appropriato per progetti che contengono codice o dati la cui riservatezza è un requisito contrattuale o legale.

AGE supporta nativamente la cifratura per destinatari multipli: il contenuto è cifrato una volta sola con una chiave di sessione, e la chiave di sessione è cifrata separatamente per ogni destinatario autorizzato. Aggiungere o rimuovere un destinatario richiede solo la modifica dell'header del file cifrato nella *commit*$_G$ successiva — il contenuto non deve essere ricitrato.

4.4.6 Gestione dei destinatari nel livello 4

La cifratura del contenuto introduce il problema della visibilità dei destinatari: come può un utente sapere se ha accesso a un progetto riservato senza tentare la decifratura? Il modello propone tre modalità configurabili nel file `.rvc_policy`:

Modalità nascosta (`recipients_mode: hidden`) — nessuna informazione sui destinatari è esposta. L'utente deve tentare la decifratura per scoprire se ha accesso. Questa modalità offre la massima riservatezza — un osservatore esterno non può determinare chi ha accesso al progetto — ma comporta un'esperienza d'uso meno immediata e non permette la costruzione di strumenti di gestione automatica dei permessi.

Modalità impronta (`recipients_mode: fingerprint`) — il file `.sig` include i fingerprint delle chiavi pubbliche dei destinatari, calcolati come *SHA-256*$_G$ della chiave. Un utente che conosce la propria chiave pubblica può calcolare il proprio fingerprint e verificare se è presente nella lista senza rivelare la propria identità a osservatori esterni. Questa modalità è il compromesso raccomandato tra usabilità e riservatezza.

Modalità pubblica (`recipients_mode: public`) — le identità complete dei destinatari sono elencate in chiaro nel file `.sig`. Questa modalità offre la massima usabilità — è immediatamente verificabile chi ha accesso — ma espone la lista dei partecipanti a qualsiasi osservatore. In contesti dove anche la lista dei partecipanti è informazione sensibile, questa modalità introduce un rischio di target enumeration: un attaccante può identificare quali soggetti hanno accesso al codice più riservato e concentrare su di essi eventuali attacchi.

Livello	Firma richiesta	Signers verificati	Verifica catena	Contenuto cifrato
0 — Aperto	No	No	No	No
1 — Autenticato	Sì	No	No	No
2 — Autorizzato	Sì	Sì	No	No
3 — Verificato	Sì	Sì	Sì	No
4 — Riservato	Sì	Sì	Sì	Sì

Confronto tra i livelli di sicurezza configurabili.

Il livello di sicurezza è definito nella prima *commit*$_G$ del progetto e non può essere abbassato. Questa proprietà è garantita dal motore di RVC: prima di accettare una nuova *commit*$_G$, il sistema verifica che il livello dichiarato nel file `.rvc_policy` sia maggiore o uguale a quello della *commit*$_G$ precedente. Qualsiasi tentativo di abbassare il livello viene rifiutato indipendentemente dall'identità del firmatario.

4.5 Gestione delle identità e ciclo di vita delle chiavi

La sicurezza di un sistema basato su crittografia asimmetrica dipende interamente dalla riservatezza delle chiavi private. Una chiave privata compromessa annulla tutte le garanzie crittografiche — autenticità, non ripudio e autorizzazione diventano prive di significato se un attaccante può produrre firme valide a nome di un utente legittimo. Il modello deve quindi definire procedure esplicite per la gestione ordinaria delle chiavi e per la risposta agli eventi straordinari.

4.5.1 Cambio chiave ordinario

Un dipendente può cambiare la propria coppia di chiavi *SSH*$_G$ in qualsiasi momento — per cambio di dispositivo, per policy aziendale di rotazione periodica o per precauzione in seguito a eventi sospetti. La procedura è la seguente:

1. Il dipendente genera una nuova coppia di chiavi con `ssh-keygen -t ed25519`.
2. Il dipendente comunica la nuova chiave pubblica al responsabile.
3. Il responsabile aggiorna il file `allowed_Dipendenti` rimuovendo la vecchia chiave pubblica e aggiungendo la nuova.
4. Il responsabile produce una *commit*$_G$ firmata con la modifica al file `allowed_Dipendenti`.

5. Dalla *commit*$_G$ successiva il dipendente firma con la nuova chiave privata.

Le *commit*$_G$ prodotte con la vecchia chiave rimangono valide — erano firmate da un'identità autorizzata al momento della firma e il file `allowed_Dipendenti` di quelle *commit*$_G$ conteneva la vecchia chiave pubblica. La storia del progetto è immutabile e ogni cambio di chiave è tracciato nella catena.

4.5.2 Revoca per compromissione

Se una chiave privata viene compromessa — rubata, esposta accidentalmente o sospettata tale — la revoca deve avvenire nel minor tempo possibile. La procedura è identica al cambio ordinario ma con priorità immediata: il responsabile aggiorna `allowed_Dipendenti` rimuovendo la chiave compromessa e produce una *commit*$_G$ di revoca esplicita che documenta l'evento.

Esiste una finestra di rischio tra il momento della compromissione e la *commit*$_G$ di revoca: durante questo intervallo un attaccante in possesso della chiave privata rubata può produrre *commit*$_G$ fraudolente che risultano valide. La dimensione di questa finestra dipende dalla rapidità con cui la compromissione viene rilevata e comunicata al responsabile. Il sistema non può eliminare questa finestra — è una limitazione strutturale di qualsiasi sistema basato su revoca — ma la minimizza richiedendo che la revoca sia operativa dalla *commit*$_G$ successiva senza procedure straordinarie.

Le *commit*$_G$ fraudolente prodotte durante la finestra di rischio rimangono nella storia e risultano valide rispetto al file `allowed_Dipendenti` di quel momento. La loro identificazione richiede un'analisi manuale della storia del progetto nel periodo sospetto.

4.5.3 Revoca offline

In un sistema distribuito non esiste un meccanismo di revoca immediata globale — la revoca è una *commit*$_G$ che deve raggiungere tutti i nodi della rete. Se un dipendente revocato tenta di fare push su una *repository*$_G$ che non ha ancora ricevuto la *commit*$_G$ di revoca, il push viene accettato localmente ma rifiutato al momento della sincronizzazione con una *repository*$_G$ aggiornata.

Questo comportamento è accettabile nel contesto d'uso di RVC: la sincronizzazione avviene tipicamente tramite push esplicito, e il rifiuto è immediato al primo tentativo di push successivo alla revoca. Se il dipendente revocato ha accesso fisico diretto alla *repository*$_G$ — ad esempio può copiare file nella cartella della repository senza passare per il motore di RVC — il problema non è più di sicurezza del sistema di versionamento ma di controllo degli accessi fisici all'infrastruttura, che è fuori dallo scope di questo modello.

4.5.4 Successione del responsabile

Se un responsabile lascia l'azienda o viene rimosso dal ruolo, l'amministratore è l'unico soggetto autorizzato a nominare un sostituto. La procedura è la seguente:

1. L'amministratore aggiorna il file `allowed_Responsabili` in `_rvc_root` rimuovendo la chiave del responsabile uscente e aggiungendo quella del nuovo responsabile.
2. L'amministratore firma la modifica con la propria chiave operativa e produce una *commit*$_G$ su `_rvc_root`.
3. Il nuovo responsabile acquisisce immediatamente i permessi sul progetto e può modificare `allowed_Dipendenti`.

Fino alla nomina del sostituto il progetto rimane in stato di attesa: i dipendenti esistenti continuano a committare normalmente, ma nessuna modifica ai permessi è possibile. Questo stato non interrompe lo sviluppo — interrompe solo la gestione amministrativa del progetto. Se il responsabile uscente era l'unico soggetto con la conoscenza operativa del progetto, il problema è organizzativo e non tecnico — il sistema garantisce la continuità delle *commit*$_G$ esistenti ma non può sostituire la conoscenza umana.

4.5.5 Compromissione della chiave dell'amministratore

La compromissione della chiave operativa dell'amministratore è lo scenario più critico del modello. L'amministratore usa la chiave master — conservata offline — per revocare la chiave operativa compromessa e nominarne una nuova. La procedura è la seguente:

1. L'amministratore recupera il dispositivo offline contenente la chiave master.
2. Genera una nuova coppia di chiavi operativa.
3. Firma la nuova chiave pubblica operativa con la chiave privata master.
4. Produce una *commit*$_G$ su `_rvc_root` che dichiara la vecchia chiave operativa non più valida e introduce la nuova, firmata con la chiave master.
5. Pubblica la nuova chiave pubblica operativa sul canale indipendente dalla *repository*$_G$.

Chiunque possieda la chiave pubblica master può verificare la legittimità della nuova chiave operativa e, da lì, ricominciare a verificare la catena di fiducia. Le *commit*$_G$ prodotte con la vecchia chiave operativa rimangono valide — erano legittime al momento della firma. Le *commit*$_G$ prodotte da un attaccante con la chiave compromessa durante la finestra di rischio sono identificabili come fraudolente tramite analisi della storia nel periodo sospetto.

La chiave master non viene mai usata nelle operazioni ordinarie — il suo utilizzo è limitato a questo scenario e alla firma iniziale della chiave operativa. Questa separazione garantisce che la compromissione della chiave operativa, per quanto critica, non comporti la perdita irreversibile del controllo della *repository*$_G$.

4.6 Gestione dei branch

I branch sono uno strumento fondamentale nello sviluppo software parallelo — permettono di isolare funzionalità, correzioni e sperimentazioni senza interferire con il lavoro principale. In un sistema di versionamento sicuro la gestione dei branch introduce scenari che vanno affrontati esplicitamente: un branch può diventare obsoleto, può essere abbandonato o può risultare compromesso a seguito di *commit*$_G$ non autorizzate.

Il principio fondamentale che governa la gestione dei branch nel modello proposto è l'**immutabilità della storia**: nessuna *commit*$_G$ viene mai cancellata o modificata retroattivamente. Qualsiasi intervento su un branch — archiviazione, chiusura o dichiarazione di compromissione — avviene aggiungendo nuove *commit*$_G$ che ne attestano lo stato, non rimuovendo quelle esistenti. Questo principio garantisce che la storia del progetto rimanga sempre verificabile nella sua interezza, inclusa la traccia degli eventi straordinari.

4.6.1 Archiviazione di branch inutili

Un branch di sviluppo che ha concluso il proprio ciclo di vita — perché la funzionalità è stata integrata nel branch principale o perché è stata abbandonata — può essere marcato come archiviato. Il responsabile produce una *commit*$_G$ firmata sul branch con un file speciale `.rvc_branch_status` all'interno dello ZIP che dichiara lo stato `archived` e la motivazione. Lo stato viene riportato anche nel campo `branch_status` del file `.sig` di quella *commit*$_G$ — in chiaro e firmato crittograficamente insieme agli altri metadati. Questo permette al motore di RVC di rilevare lo stato del branch leggendo esclusivamente il `.sig`, senza dover decifrare lo ZIP, garantendo il corretto funzionamento anche per i progetti a livello 4.

Il motore tratta i branch archiviati come sola lettura — è possibile leggerne la storia ma non aggiungere nuove *commit*$_G$. L'archiviazione è reversibile: il responsabile o l'amministratore può produrre una nuova *commit*$_G$ che riporta lo stato a `active`. Questa operazione è tracciata nella catena e richiede una firma autorizzata.

4.6.2 Chiusura di branch compromessi

Un branch compromesso è un branch su cui è stata prodotta una o più *commit*$_G$ fraudolente o non autorizzate — ad esempio durante la finestra di rischio successiva alla compromissione di una chiave privata. La gestione di questo scenario segue una procedura in due fasi.

Nella prima fase il branch compromesso viene dichiarato tale tramite una *commit*$_G$ firmata dal responsabile o dall'amministratore. Il file `.rvc_branch_status` all'interno dello ZIP dichiara lo stato `compromised`, il riferimento all'identificativo della prima *commit*$_G$

sospetta e la motivazione. Lo stato `compromised` viene riportato anche nel campo `branch_status` del file `.sig` — in chiaro e firmato — in modo che il motore possa imporre il blocco immediato del branch senza dover decifrare il contenuto, indipendentemente dal livello di sicurezza del progetto. Il branch viene immediatamente bloccato — nessuna nuova *commit*$_G$ è accettata. Le *commit*$_G$ fraudolente rimangono nella storia e sono visibili, ma il branch è marcato come non affidabile.

Nella seconda fase viene creato un nuovo branch pulito a partire dall'ultima *commit*$_G$ verificata come integra prima della compromissione. Lo sviluppo riprende sul nuovo branch. Il branch compromesso rimane nella *repository*$_G$ come evidenza dell'incidente — la sua storia è verificabile e costituisce la prova crittografica di cosa è accaduto e quando.

Questa procedura garantisce che la risposta a una compromissione non introduca ambiguità nella storia del progetto. Un approccio alternativo — cancellare le *commit*$_G$ fraudolente — comprometterebbe l'integrità della catena e renderebbe impossibile distinguere una storia ripulita da una storia alterata da un attaccante.

4.6.3 Branch e permessi

Come definito nella sezione sulla gerarchia di fiducia, i permessi sono per progetto e non per branch. Un dipendente autorizzato su un progetto può committare su qualsiasi branch di quel progetto. La disciplina sui branch — lavorare su branch di sviluppo e fare merge sul branch principale solo quando il codice è verificato — è una convenzione operativa del team, non un vincolo tecnico imposto dal sistema.

Questa scelta è deliberata: introdurre permessi per branch aggiungerebbe complessità gestionale significativa senza un corrispondente aumento delle garanzie di sicurezza. La sicurezza del sistema si basa sull'autenticità delle *commit*$_G$, non sulla restrizione dei branch su cui è possibile scrivere. Un dipendente che produce una *commit*$_G$ non autorizzata — ad esempio direttamente sul branch principale saltando il processo di verifica — è comunque identificabile tramite la firma crittografica e la sua azione è permanentemente tracciata nella storia del progetto.

4.7 Gap analysis

Capitolo 5

Simulazione scenari di attacco

In questo capitolo effettuo l'analisi degli utenti, sviluppo le user stories e compongo la lista dei requisiti dividendoli per tipologia e necessità.

Capitolo 6

Miglioramenti implementati

In questo capitolo effettuo l'analisi degli utenti, sviluppo le user stories e compongo la lista dei requisiti dividendoli per tipologia e necessità.

Capitolo 7

Conclusioni

In questo capitolo traggo le conclusioni sul progetto.

7.1 Consuntivo finale

Una volta terminato il progetto ho redatto il consuntivo orario finale nella 7.1 che suddivide in maniera approssimata le ore dedicate alle varie fasi.

Fase	**Ore**
Onboarding del progetto	5
Analisi dei requisiti	30
...	...
Totale	320

Consuntivo orario finale.

7.2 Raggiungimento degli obiettivi

7.3 Requisiti soddisfatti

Arrivato alla fine del progetto ho implementato...

Tipo	Mandatory	Desirable	Optional	Somma
Functional	0/2	0/0	0/0	0/2
Qualitative	0/1	0/1	0/1	0/3
Constraint	0/1	0/0	0/0	0/1
Totale	**0/4**	**0/1**	**0/1**	**0/6**

Riepilogo dei requisiti soddisfatti.

7.4 Rischi occorsi e mitigati

I rischi emersi durante lo stage sono riportati in 7.3.

Descrizione	Mitigazione
R1 – Descrizione del rischio	Soluzione

Rischi occorsi con la loro mitigazione.

7.5 Valutazione personale

Glossario

AGE – Actually Good Encryption: Strumento moderno per la cifratura di file. Utilizza algoritmi crittografici contemporanei come X25519 e ChaCha20-Poly1305, con un'interfaccia volutamente semplice. 3., 4., 8., 9., 11., 14., 15., 25., 26., 33.

Black-box: Approccio di analisi della sicurezza condotto senza accesso al codice sorgente, osservando esclusivamente il comportamento esterno del sistema in risposta agli input forniti. 5.

Chiave privata: Componente segreta di una coppia di chiavi asimmetriche. Deve essere custodita esclusivamente dal proprietario e viene utilizzata per decifrare messaggi o produrre firme digitali. 21., 24.

Chiave pubblica: Componente pubblica di una coppia di chiavi asimmetriche. Può essere distribuita liberamente e viene utilizzata per cifrare messaggi o verificare firme digitali. 21.

Commit: Unità atomica di modifica in un sistema di versionamento. Rappresenta uno snapshot dello stato dei file in un determinato momento, accompagnato da metadati quali autore, data e messaggio descrittivo. v, 4., 16., 17., 18., 19., 20., 21., 22., 23., 24., 25., 26., 27., 28., 29., 30., 31., 32., 33., 34., 35., 36., 37., 38.

CPL – CodePainter Language: Linguaggio di programmazione proprietario sviluppato da Zucchetti S.p.A. Interpretato, tipizzato staticamente, con sintassi simile al Pascal. Utilizzato per lo sviluppo di RVC e dei prodotti software Zucchetti. 4., 5., 8., 9., 17., 19.

Ed25519: Algoritmo di firma digitale basato sulla curva ellittica Curve25519. Produce chiavi di 256 bit con un livello di sicurezza equivalente a RSA con chiavi da 3000 bit, risultando più efficiente e compatto. 8., 13., 14.

Firma digitale: Meccanismo crittografico che garantisce autenticità e integrità di un documento. Prodotta con la chiave privata del firmatario, può essere verificata da chiunque possieda la chiave pubblica corrispondente. 21.

Git: Sistema di versionamento distribuito open source, creato da Linus Torvalds nel 2005. Garantisce l'integrità della storia tramite una catena di hash crittografici. v, 2., 16., 17., 22., 23.

Hash: Funzione matematica che trasforma un dato di dimensione arbitraria in una stringa di lunghezza fissa. Una minima modifica dell'input produce un output completamente diverso, garantendo l'integrità dei dati. v, vi, 2., 9., 12., 17., 18., 21., 23., 24., 33.

JIT – Just-In-Time: Tecnica di compilazione che traduce il codice intermedio in codice macchina durante l'esecuzione, migliorando le prestazioni rispetto alla sola interpretazione. 19.

Manifest: Nel contesto di RVC, file che descrive lo stato di tutti i file tracciati in un progetto in un dato momento: nomi, dimensioni, hash e versione di ogni file. 19.

Namespace: Nel contesto della firma SSH, stringa che identifica il contesto d'uso di una firma e previene attacchi di riutilizzo: una firma prodotta con un certo namespace non può essere usata in un contesto diverso. 14.

Passphrase: Sequenza di parole o caratteri utilizzata per proteggere una chiave privata. Più lunga e complessa di una password tradizionale, viene richiesta ogni volta che si utilizza la chiave. 14.

Repository: Archivio che contiene la storia completa delle modifiche di un progetto sotto controllo versione, inclusi tutti i commit, i branch e i tag. v, 8., 9., 16., 17., 19., 20., 22., 24., 27., 28., 29., 30., 31., 32., 35., 36., 38.

RVC – Repositoryless Version Control: Sistema di versionamento distribuito sviluppato da Zucchetti S.p.A. come alternativa a Git. Garantisce l'integrità dei contenuti tramite verifica crittografica degli hash di ogni commit, permettendo a qualsiasi utente di accertare autonomamente l'autenticità della repository ricevuta. v, vi, vii, xi, 2., 3., 4., 5., 8., 9., 11., 14., 17., 18., 19., 23.

SHA-256 – Secure Hash Algorithm 256: Funzione di hash crittografica che produce un'impronta digitale di 256 bit. Ampiamente utilizzata per verificare l'integrità dei dati grazie alla sua resistenza alle collisioni e alla non invertibilità. 21., 22., 23., 33.

SSH – Secure Shell: Protocollo crittografico per la comunicazione sicura su reti non affidabili. Utilizza crittografia asimmetrica per l'autenticazione e la cifratura del canale. v, vi, 2., 3., 4., 8., 9., 11., 13., 14., 17., 18., 19., 22., 23., 24., 32., 34.

VCS – Version Control System: Sistema software che registra le modifiche ai file nel tempo, permettendo di recuperare versioni precedenti, confrontare cambiamenti e coordinare il lavoro di più sviluppatori. 15., 16.

White-box: Approccio di analisi della sicurezza con piena visibilità del codice sorgente e dell'architettura interna del sistema. Si contrappone all'analisi black-box, condotta senza accesso ai sorgenti. 5.